Phase 7: Merging and Iterative Merge Sort — Full Structured Beginner Note

1. Current Progress Before Phase 7

Before Phase 7, you have already learned six important phases of sorting.

Phase 1: Sorting Foundations

In Phase 1, you learned the basic language used to understand sorting algorithms.

You learned words like:

Term	Simple meaning
Sorting	Arranging data in order
Ascending order	Smallest to largest
Descending order	Largest to smallest
Comparison	Checking two values to decide their order
Swap	Exchanging two values
Shift	Moving values to make space
Adaptive	Faster when data is already sorted or nearly sorted
Extra memory	Additional space used by an algorithm
Stable	Equal values keep their original relative order
Big O	Describes how algorithm work grows
O(n)	Work grows directly with input size
$O(n^2)$	Work grows very quickly, often because of nested loops
O(n log n)	Usually a fast class for comparison sorting
Pointer	A variable that stores an address

The most important lesson from Phase 1 was:

> Do not judge a sorting algorithm only by asking, “Does it sort?”

A better question is:

How does it sort, how much time does it take, how much memory does it use, and does it preserve duplicate order?

Phase 2: Bubble Sort

In Phase 2, you learned Bubble Sort.

Bubble Sort works by comparing neighboring elements and swapping them if they are in the wrong order.

Example:

```
8 5
```

Since:

```
8 > 5
```

Bubble Sort swaps them:

```
5 8
```

Important points from Bubble Sort:

- It compares adjacent elements.
- It uses swaps.
- One full scan is called a pass.
- After each pass, one large value is fixed at the end.
- After `k` passes, the `k` largest values are fixed at the end.
- It can be adaptive if we use a `flag`.
- It is stable if we use `>` instead of `>=`.
- Best-case time is `O(n)` with the adaptive flag.
- Worst-case time is `O(n²)`.
- Space complexity is `O(1)`.

Phase 3: Insertion Sort

In Phase 3, you learned Insertion Sort.

Insertion Sort builds a sorted left part.

Example:

```
[5 8] 7 3 2
```

The left part `[5 8]` is sorted.

Now we insert `7` into the correct position:

```
[5 7 8] 3 2
```

Important points from Insertion Sort:

- It inserts one value into a sorted left part.
- It starts from index `1` because the first element alone is already sorted.
- It stores the selected value in `x`.
- It shifts bigger values to the right.
- It is naturally adaptive.
- It is stable if we use `>` instead of `>=`.
- Best-case time is `O(n)`.
- Worst-case time is `O(n²)`.
- Space complexity is `O(1)`.

Phase 4: Selection Sort

In Phase 4, you learned Selection Sort.

Selection Sort selects the smallest value from the unsorted part and puts it at the front.

Example:

```
8 6 3 2 5 4
```

The smallest value is `2`, so Selection Sort swaps `2` with the first value:

```
2 6 3 8 5 4
```

Important points from Selection Sort:

- It selects the minimum value.
- It fills the array from left to right.
- It uses `i`, `j`, and `k`.
- `i` is the position being filled.
- `j` scans the unsorted part.

- `k` stores the index of the smallest value found so far.
- It swaps only after the inner scan finishes.
- It performs fewer swaps than Bubble Sort.
- It is not adaptive.
- It is usually not stable.
- Best-case time is `O(n²)`.
- Worst-case time is `O(n²)`.
- Space complexity is `O(1)`.
- After `k` passes, the `k` smallest values are fixed at the front.

Phase 5: Quick Sort Partitioning

In Phase 5, you learned the heart of Quick Sort: partitioning.

Partitioning means:

> Choose one value, called the pivot, and place it in its final sorted position.

Example:

```
50  70  60  90  40  80  10  20  30  ∞
```

The pivot is `50`.

After partitioning:

```
40  30  20  10  |  50  |  80  90  60  70
```

Now `50` is in its final sorted position.

But the left side is not fully sorted:

```
40  30  20  10
```

And the right side is not fully sorted:

```
80  90  60  70
```

So partitioning alone does not fully sort the array.

Phase 6: Quick Sort Recursion and Complexity

In Phase 6, you completed Quick Sort.

You learned that Quick Sort works by doing this:

1. Partition the array.
2. Sort the left side recursively.
3. Sort the right side recursively.

Simple memory trick:

> Partition, sort left, sort right.

Important points from Quick Sort:

- Quick Sort uses divide and conquer.
- It chooses a pivot.
- Partitioning places the pivot in its correct position.
- Recursion sorts the smaller parts.
- Best-case time is `O(n log n)`.
- Average-case time is `O(n log n)`.
- Worst-case time is `O(n²)`.
- First-element pivot Quick Sort can be bad for already sorted data.
- It is usually not stable.
- It is in-place in terms of array storage, but recursion uses stack memory.

---

2. Why Phase 7 Comes After Quick Sort

Quick Sort taught you one divide-and-conquer idea:

> Divide by partitioning around a pivot.

Phase 7 begins a new divide-and-conquer idea:

> Build sorted lists by merging smaller sorted lists.

This is the foundation of Merge Sort.

Before learning Recursive Merge Sort in Phase 8, you must first understand the most important operation:

> Merging.

Merge Sort depends completely on merging.

If you do not understand merging clearly, Recursive Merge Sort will feel confusing.

So Phase 7 teaches:

- Merging two sorted arrays.
- Merging two sorted sections inside one array.
- Using an auxiliary array.
- 2-way merging.
- m-way merging.
- Iterative Merge Sort.
- Passes of Iterative Merge Sort.
- Complete C implementation.
- Time and space complexity.
- Common beginner mistakes.

---

3. Main Goal of Phase 7

The main goal of Phase 7 is to understand this idea:

> If two lists are already sorted, we can combine them into one bigger sorted list.

This operation is called **merging**.

Simple memory trick:

> Sorted pieces combine.

Another memory trick:

> Merge small sorted parts into bigger sorted parts.

---

4. What Is Merging?

Merging means combining two already sorted lists into one sorted list.

Example:

```
A = [2, 6, 10]
B = [3, 7]
```

Both arrays are already sorted.

After merging:

```
C = [2, 3, 6, 7, 10]
```

The result is one bigger sorted array.

---

5. The Most Important Rule of Merging

Merging works correctly only when the input parts are already sorted.

Correct example:

```
A = [2, 6, 10]
B = [3, 7]
```

Both are sorted, so merging works.

Wrong example:

```
A = [6, 2, 10]
B = [7, 3]
```

These are not sorted.

If we try to merge these directly, we should not expect the result to be correctly sorted.

Important beginner rule:

> Merging does not magically sort random chunks. It combines sorted chunks.

---

6. Real-Life Analogy for Merging

Imagine two students have already arranged their books from shortest to tallest.

Student A has:

```
2 6 10
```

Student B has:

```
3 7
```

Now you want to create one final sorted row.

You only need to compare the first remaining book from each student.

Compare `2` and `3`.

Take `2`.

Compare `6` and `3`.

Take `3`.

Compare `6` and `7`.

Take `6`.

Compare `10` and `7`.

Take `7`.

Now Student B has no books left.

Copy the remaining book from Student A.

Final result:

```
2 3 6 7 10
```

That is merging.

---

7. Merging Two Separate Sorted Arrays

Suppose we have two separate sorted arrays:

```
A = [2, 6, 10]
B = [3, 7]
```

We want to create a third sorted array:

```
C = []
```

We use three index variables:

```
i -> reads A
j -> reads B
k -> writes into C
```

Meaning of the variables

Variable	Meaning
`A[]`	First sorted array
`B[]`	Second sorted array
`C[]`	Final merged array
`i`	Index used to read array `A`
`j`	Index used to read array `B`
`k`	Index used to write into array `C`
`m`	Number of elements in `A`
`n`	Number of elements in `B`

---

8. Dry Run: Merging Two Separate Sorted Arrays

We want to merge:

```
A = [2, 6, 10]
B = [3, 7]
```

At the start:

```
A = [2, 6, 10]
     i

B = [3, 7]
     j
```

```
C = []
      k
```

Step 1: Compare A[i] and B[j]

Compare:

```
2  and  3
```

`2` is smaller.

Copy `2` into `C`.

```
C = [2]
```

Move `i` forward.

Step 2: Compare A[i] and B[j]

Now compare:

```
6  and  3
```

`3` is smaller.

Copy `3` into `C`.

```
C = [2, 3]
```

Move `j` forward.

Step 3: Compare A[i] and B[j]

Now compare:

```
6  and  7
```

`6` is smaller.

Copy `6` into `C`.

```
C = [2, 3, 6]
```

Move `i` forward.

Step 4: Compare A[i] and B[j]

Now compare:

```
10 and 7
```

`7` is smaller.

Copy `7` into `C`.

```
C = [2, 3, 6, 7]
```

Move `j` forward.

Now array `B` is finished.

Step 5: Copy leftover values

Array `B` has no values left.

Array `A` still has:

```
10
```

Copy the leftover value into `C`.

```
C = [2, 3, 6, 7, 10]
```

Final merged array:

```
[2, 3, 6, 7, 10]
```

---

9. Why Leftover Copying Is Needed

During merging, one array may finish before the other.

Example:

```
A = [2, 6, 10]
B = [3, 7]
```

Array `B` finishes first.

But array `A` still has `10` left.

If we forget to copy leftovers, the result becomes incomplete:

```
Wrong result: [2, 3, 6, 7]
```

Correct result:

```
[2, 3, 6, 7, 10]
```

Important rule:

> After one list finishes, copy all remaining values from the other list.

Why is this safe?

Because the remaining values are already sorted.

---

10. Code: Merging Two Separate Sorted Arrays

```c
#include <stdio.h>

void MergeTwoArrays(int A[], int B[], int m, int n, int C[]) {
    int i = 0, j = 0, k = 0;
```

```
    while (i < m && j < n) {
        if (A[i] < B[j]) {
            C[k++] = A[i++];
        } else {
            C[k++] = B[j++];
        }
    }

    while (i < m) {
        C[k++] = A[i++];
    }

    while (j < n) {
        C[k++] = B[j++];
    }
}
```

---

11. Explanation of MergeTwoArrays Code

Function header

```
void MergeTwoArrays(int A[], int B[], int m, int n, int C[])
```

This function receives:

Parameter	Meaning
`A[]`	First sorted array
`B[]`	Second sorted array
`m`	Number of elements in `A`
`n`	Number of elements in `B`
`C[]`	Output array where merged values are stored

Initialize indexes

```
int i = 0, j = 0, k = 0;
```

This means:

- `i` starts at the first element of `A`.
- `j` starts at the first element of `B`.
- `k` starts at the first position of `C`.

Main comparison loop

```
while (i < m && j < n) {
```

This loop continues while both arrays still have elements.

The condition means:

- `i < m`: array `A` still has values.
- `j < n`: array `B` still has values.

Both must be true.

Copy the smaller value

```
if (A[i] < B[j]) {
    C[k++] = A[i++];
} else {
    C[k++] = B[j++];
}
```

This compares the current values of `A` and `B`.

If `A[i]` is smaller, copy `A[i]` into `C[k]`.

Otherwise, copy `B[j]` into `C[k]`.

Meaning of k++ and i++

This line:

```
C[k++] = A[i++];
```

means:

1. Copy `A[i]` into `C[k]`.
2. Move `k` to the next position.
3. Move `i` to the next position.

So it is a short form of:

```
C[k] = A[i];
k++;
i++;
```

Copy leftovers from A

```
while (i < m) {
    C[k++] = A[i++];
}
```

If array `B` finishes first, this copies the remaining values from `A`.

Copy leftovers from B

```
while (j < n) {
    C[k++] = B[j++];
}
```

If array `A` finishes first, this copies the remaining values from `B`.

---

12. Complete Program: Merging Two Separate Sorted Arrays

```
#include <stdio.h>

void MergeTwoArrays(int A[], int B[], int m, int n, int C[]) {
    int i = 0, j = 0, k = 0;

    while (i < m && j < n) {
        if (A[i] < B[j]) {
            C[k++] = A[i++];
        } else {
```

```
            C[k++] = B[j++];
        }
    }

    while (i < m) {
        C[k++] = A[i++];
    }

    while (j < n) {
        C[k++] = B[j++];
    }
}

int main() {
    int A[] = {2, 6, 10};
    int B[] = {3, 7};
    int C[5];

    int m = sizeof(A) / sizeof(A[0]);
    int n = sizeof(B) / sizeof(B[0]);

    MergeTwoArrays(A, B, m, n, C);

    printf("Merged array: ");
    for (int i = 0; i < m + n; i++) {
        printf("%d ", C[i]);
    }

    printf("\n");
    return 0;
}
```

Expected output:

```
Merged array: 2 3 6 7 10
```

---

13. Merging Two Sorted Sections Inside One Array

Now we move to a more important version of merging.

Sometimes the two sorted lists are inside the same array.

Example:

```
A = [2, 6, 10, 3, 7, 12]
```

At first, this whole array does not look sorted.

But it has two sorted sections:

```
[2, 6, 10] [3, 7, 12]
```

The first section is sorted.

The second section is sorted.

So we can merge them.

---

14. low, mid, and high

To merge two sorted sections inside one array, we need three positions:

```
low
mid
high
```

For this example:

```
A = [2, 6, 10, 3, 7, 12]
```

We use:

```
low = 0
mid = 2
high = 5
```

So:

```
left section  = A[low] to A[mid]
right section = A[mid + 1] to A[high]
```

That means:

```
left section  = A[0] to A[2] = [2, 6, 10]
right section = A[3] to A[5] = [3, 7, 12]
```

---

15. Meaning of low, mid, and high

Variable	Meaning
`low`	Starting index of the first sorted section
`mid`	Ending index of the first sorted section
`mid + 1`	Starting index of the second sorted section
`high`	Ending index of the second sorted section

Visual example:

```
A = [2, 6, 10, 3, 7, 12]
     low   mid  mid+1   high
```

More clearly:

```
Index:  0  1   2   3  4   5
Array:  2  6  10   3  7  12
        low   mid  ^      high
                   mid+1
```

---

16. Why We Need an Auxiliary Array B

When merging two sections inside the same array, we should not directly overwrite the original array too early.

Why?

Because we may destroy values before we use them.

Example:

```
A = [2, 6, 10, 3, 7, 12]
```

If we start writing the merged result directly into `A`, we may overwrite a value that we still need to compare later.

So we use a temporary array:

```
B[]
```

We first merge into `B`.

Then we copy from `B` back into `A`.

Important rule:

> Merge safely into a temporary array, then copy back.

---

17. Code: Merging Two Sorted Sections in One Array

```c
void Merge(int A[], int low, int mid, int high) {
    int i = low;
    int j = mid + 1;
    int k = low;
    int B[100];

    while (i <= mid && j <= high) {
        if (A[i] < A[j]) {
            B[k++] = A[i++];
        } else {
            B[k++] = A[j++];
        }
    }

    while (i <= mid) {
        B[k++] = A[i++];
    }

    while (j <= high) {
        B[k++] = A[j++];
    }

    for (i = low; i <= high; i++) {
        A[i] = B[i];
    }
}
```

18. Explanation of Merge Function

Function header

```
void Merge(int A[], int low, int mid, int high)
```

This function merges two sorted sections inside the same array.

Parameter	Meaning
`A[]`	Array containing the two sorted sections
`low`	Start of the left section
`mid`	End of the left section
`high`	End of the right section

Initialize i

```
int i = low;
```

`i` starts at the beginning of the left section.

Initialize j

```
int j = mid + 1;
```

`j` starts at the beginning of the right section.

Initialize k

```
int k = low;
```

`k` starts writing into the same index range inside temporary array `B`.

Temporary array

```
int B[100];
```

`B` temporarily stores the merged result.

Beginner note:

`B[100]` means this code can safely work only for arrays whose needed index range fits within 100 elements.

For serious programs, we should create `B` based on the actual size of the input.

Main merging loop

```
while (i <= mid && j <= high)
```

This means:

- Continue while the left section still has values.
- Continue while the right section still has values.

Compare and copy smaller value

```
if (A[i] < A[j]) {
    B[k++] = A[i++];
} else {
    B[k++] = A[j++];
}
```

This compares the current value of the left section and the current value of the right section.

The smaller value is copied into `B`.

Copy leftovers from left section

```
while (i <= mid) {
    B[k++] = A[i++];
}
```

If the right section finishes first, copy the remaining left-section values.

Copy leftovers from right section

```
while (j <= high) {
    B[k++] = A[j++];
}
```

If the left section finishes first, copy the remaining right-section values.

Copy B back to A

```
for (i = low; i <= high; i++) {
    A[i] = B[i];
}
```

This step is very important.

The merged values are currently in `B`.

But the original array is `A`.

So we must copy the sorted merged section from `B` back into `A`.

Without this step, `A` will not be updated correctly.

---

19. Dry Run: Merging Two Sections in One Array

Let:

```
A = [2, 6, 10, 3, 7, 12]
```

We have:

```
low = 0
mid = 2
high = 5
```

So:

```
Left section  = [2, 6, 10]
Right section = [3, 7, 12]
```

Start:

```
i = 0, points to 2
j = 3, points to 3
k = 0
```

Step 1

Compare:

```
2 and 3
```

`2` is smaller.

Copy `2` to `B[0]`.

```
B = [2]
```

Move `i` and `k` forward.

Step 2

Compare:

```
6 and 3
```

`3` is smaller.

Copy `3` to `B[1]`.

```
B = [2, 3]
```

Move `j` and `k` forward.

Step 3

Compare:

```
6 and 7
```

`6` is smaller.

Copy `6` to `B[2]`.

```
B = [2, 3, 6]
```

Move `i` and `k` forward.

Step 4

Compare:

```
10 and 7
```

`7` is smaller.

Copy `7` to `B[3]`.

```
B = [2, 3, 6, 7]
```

Move `j` and `k` forward.

Step 5

Compare:

```
10 and 12
```

`10` is smaller.

Copy `10` to `B[4]`.

```
B = [2, 3, 6, 7, 10]
```

Move `i` and `k` forward.

Now the left section is finished.

Step 6: Copy leftovers

The right section still has:

```
12
```

Copy it:

```
B = [2, 3, 6, 7, 10, 12]
```

Step 7: Copy B back to A

Now copy:

```
B[0] to A[0]
B[1] to A[1]
B[2] to A[2]
B[3] to A[3]
B[4] to A[4]
B[5] to A[5]
```

Final array:

```
A = [2, 3, 6, 7, 10, 12]
```

---

20. Complete Program: Merging Two Sorted Sections

```c
#include <stdio.h>

void Merge(int A[], int low, int mid, int high) {
    int i = low;
    int j = mid + 1;
    int k = low;
    int B[100];

    while (i <= mid && j <= high) {
        if (A[i] < A[j]) {
            B[k++] = A[i++];
        } else {
            B[k++] = A[j++];
        }
    }

    while (i <= mid) {
        B[k++] = A[i++];
    }

    while (j <= high) {
        B[k++] = A[j++];
    }

    for (i = low; i <= high; i++) {
        A[i] = B[i];
    }
}

int main() {
    int A[] = {2, 6, 10, 3, 7, 12};
    int n = sizeof(A) / sizeof(A[0]);

    Merge(A, 0, 2, 5);

    printf("Merged array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", A[i]);
    }
```

```
    printf("\n");
    return 0;
}
```

Expected output:

```
Merged array: 2 3 6 7 10 12
```

---

21. What Is 2-Way Merging?

2-way merging means merging two sorted lists at a time.

Example:

```
[3, 8] and [4, 7]
```

Merge them into:

```
[3, 4, 7, 8]
```

It is called 2-way because we combine two lists.

Simple meaning:

> 2-way merging = merge two sorted parts.

---

22. What Is m-Way Merging?

m-way merging means merging many sorted lists at the same time.

Example:

```
[1, 9], [2, 6], [3, 7], [4, 8]
```

Here, there are four sorted lists.

If we merge all of them together at once, that is m-way merging.

Simple meaning:

> m-way merging = merge many sorted parts.

---

23. 2-Way Merging vs m-Way Merging

Type	Meaning	Beginner difficulty
2-way merging	Merge two sorted lists	Easier
m-way merging	Merge many sorted lists	More difficult

Why is m-way merging harder?

Because we must compare the first value of many lists at the same time.

In 2-way merging, we compare only two values:

```
A[i] and B[j]
```

In m-way merging, we may need to compare many front values.

Phase 7 focuses mainly on 2-way merging because it is easier to program and becomes the base of Merge Sort.

---

24. What Is Iterative Merge Sort?

Iterative Merge Sort is a sorting algorithm that sorts an array by repeatedly merging sorted parts.

It does not use recursion.

Instead, it uses loops.

The main idea is:

> Start with small sorted parts and repeatedly merge them into bigger sorted parts.

The smallest sorted part is one element.

Why?

Because a single element is always sorted.

Example:

```
[8]
```

This is already sorted.

So for an array:

```
[8, 3, 7, 4, 9, 2, 6, 5]
```

We can first think of it as:

```
[8] [3] [7] [4] [9] [2] [6] [5]
```

Each one-element list is sorted.

Now Iterative Merge Sort merges them step by step.

---

25. Main Idea of Iterative Merge Sort

Iterative Merge Sort works like this:

1. Treat every single element as a sorted list.
2. Merge pairs of size `1` to make sorted lists of size `2`.
3. Merge pairs of size `2` to make sorted lists of size `4`.
4. Merge pairs of size `4` to make sorted lists of size `8`.
5. Continue until the whole array is sorted.

Memory trick:

> 1 becomes 2, 2 becomes 4, 4 becomes 8.

Or:

> Small sorted groups become bigger sorted groups.

---

26. Iterative Merge Sort Example

Let us sort:

```
[8, 3, 7, 4, 9, 2, 6, 5]
```

Final sorted result should be:

```
[2, 3, 4, 5, 6, 7, 8, 9]
```

At the beginning, think of each value as a sorted list:

```
[8] [3] [7] [4] [9] [2] [6] [5]
```

---

27. Pass 1: Merge Size 1 Into Size 2

Original:

```
[8] [3] [7] [4] [9] [2] [6] [5]
```

Merge `[8]` and `[3]`:

```
[3, 8]
```

Merge `[7]` and `[4]`:

```
[4, 7]
```

Merge `[9]` and `[2]`:

```
[2, 9]
```

Merge `[6]` and `[5]`:

```
[5, 6]
```

After pass 1:

```
[3, 8, 4, 7, 2, 9, 5, 6]
```

Now every group of 2 is sorted:

```
[3, 8] [4, 7] [2, 9] [5, 6]
```

---

28. Pass 2: Merge Size 2 Into Size 4

Now merge these groups:

```
[3, 8] and [4, 7]
```

Result:

```
[3, 4, 7, 8]
```

Now merge:

```
[2, 9] and [5, 6]
```

Result:

```
[2, 5, 6, 9]
```

After pass 2:

```
[3, 4, 7, 8, 2, 5, 6, 9]
```

Now every group of 4 is sorted:

```
[3, 4, 7, 8] [2, 5, 6, 9]
```

---

29. Pass 3: Merge Size 4 Into Size 8

Now merge:

```
[3, 4, 7, 8] and [2, 5, 6, 9]
```

Result:

```
[2, 3, 4, 5, 6, 7, 8, 9]
```

Now the whole array is sorted.

Final result:

```
[2, 3, 4, 5, 6, 7, 8, 9]
```

---

30. Summary of Iterative Merge Sort Passes

Original array:

```
[8, 3, 7, 4, 9, 2, 6, 5]
```

After pass 1:

```
[3, 8, 4, 7, 2, 9, 5, 6]
```

After pass 2:

```
[3, 4, 7, 8, 2, 5, 6, 9]
```

After pass 3:

```
[2, 3, 4, 5, 6, 7, 8, 9]
```

---

31. What Does p Mean in Iterative Merge Sort?

In the Iterative Merge Sort code, `p` means the size of the current merge group.

`p` keeps doubling:

```
2, 4, 8, 16, ...
```

When p = 2

```
p = 2
```

We merge pairs of one-element lists.

Example:

```
[8] and [3] -> [3, 8]
```

So `p = 2` creates sorted groups of size 2.

When p = 4

```
p = 4
```

We merge pairs of two-element lists.

Example:

```
[3, 8] and [4, 7] -> [3, 4, 7, 8]
```

So `p = 4` creates sorted groups of size 4.

When p = 8

```
p = 8
```

We merge pairs of four-element lists.

Example:

```
[3, 4, 7, 8] and [2, 5, 6, 9]
```

Result:

```
[2, 3, 4, 5, 6, 7, 8, 9]
```

So `p = 8` creates sorted groups of size 8.

---

32. Why p Doubles Each Time

In Iterative Merge Sort, every pass doubles the size of sorted groups.

Pass	What happens	Group size after pass
Pass 1	Merge 1-element groups	2
Pass 2	Merge 2-element groups	4
Pass 3	Merge 4-element groups	8
Pass 4	Merge 8-element groups	16

That is why the loop uses:

```
p = p * 2
```

This doubling is also why Merge Sort has about $\log_2(n)$ passes.

---

33. Iterative Merge Sort Code

```c
#include <stdio.h>

void Merge(int A[], int low, int mid, int high) {
    int i = low;
    int j = mid + 1;
    int k = low;
    int B[100];
```

```
    while (i <= mid && j <= high) {
        if (A[i] < A[j]) {
            B[k++] = A[i++];
        } else {
            B[k++] = A[j++];
        }
    }

    while (i <= mid) {
        B[k++] = A[i++];
    }

    while (j <= high) {
        B[k++] = A[j++];
    }

    for (i = low; i <= high; i++) {
        A[i] = B[i];
    }
}

void IterativeMergeSort(int A[], int n) {
    int p, i, low, mid, high;

    for (p = 2; p <= n; p = p * 2) {
        for (i = 0; i + p - 1 < n; i = i + p) {
            low = i;
            high = i + p - 1;
            mid = (low + high) / 2;
            Merge(A, low, mid, high);
        }
    }

    if (p / 2 < n) {
        Merge(A, 0, p / 2 - 1, n - 1);
    }
}

int main() {
    int A[] = {8, 3, 7, 4, 9, 2, 6, 5};
    int n = sizeof(A) / sizeof(A[0]);

    IterativeMergeSort(A, n);

    for (int i = 0; i < n; i++) {
        printf("%d ", A[i]);
    }
```

```
    printf("\n");
    return 0;
}
```

Expected output:

```
2 3 4 5 6 7 8 9
```

---

34. Explanation of IterativeMergeSort Function

Function header

```
void IterativeMergeSort(int A[], int n)
```

This function receives:

Parameter	Meaning
`A[]`	Array to sort
`n`	Number of elements in the array

Variables

```
int p, i, low, mid, high;
```

Variable	Meaning
`p`	Current merge group size
`i`	Starting index of each group being merged
`low`	Start of the left section
`mid`	End of the left section
`high`	End of the right section

Outer loop

```
for (p = 2; p <= n; p = p * 2)
```

This controls the group size.

When `p = 2`, we merge groups into size 2.

When `p = 4`, we merge groups into size 4.

When `p = 8`, we merge groups into size 8.

So `p` doubles each time.

Inner loop

```
for (i = 0; i + p - 1 < n; i = i + p)
```

This moves through the array group by group.

For each group of size `p`, it calculates:

```
low = i;
high = i + p - 1;
mid = (low + high) / 2;
```

Then it calls:

```
Merge(A, low, mid, high);
```

This merges the two sorted halves inside that group.

---

35. Meaning of low, mid, high Inside Iterative Merge Sort

Suppose:

```
A = [8, 3, 7, 4, 9, 2, 6, 5]
```

When `p = 2`, the first group is:

```
[8, 3]
```

For this group:

```
low = 0
high = 1
mid = 0
```

Left section:

```
A[0] to A[0] = [8]
```

Right section:

```
A[1] to A[1] = [3]
```

After merging:

```
[3, 8]
```

When `p = 4`, the first group is:

```
[3, 8, 4, 7]
```

For this group:

```
low = 0
high = 3
mid = 1
```

Left section:

```
A[0] to A[1] = [3, 8]
```

Right section:

```
A[2] to A[3] = [4, 7]
```

After merging:

```
[3, 4, 7, 8]
```

---

36. Why the Final if Condition Is Needed

The code has this part:

```
if (p / 2 < n) {
    Merge(A, 0, p / 2 - 1, n - 1);
}
```

This handles leftover elements when the number of elements is not a perfect power of 2.

A perfect power of 2 means numbers like:

```
2, 4, 8, 16, 32
```

If the array size is exactly 8, the passes work neatly:

```
1 -> 2 -> 4 -> 8
```

But if the array size is not a power of 2, there may be leftover sorted parts at the end.

Example:

```
n = 10
```

The group sizes may not divide the array perfectly.

So the final `if` makes sure the remaining part is merged into the full sorted array.

Beginner meaning:

The final if handles arrays whose size does not fit perfectly into 2, 4, 8, 16 groups.

---

37. Complete Dry Run With p Values

Array:

```
[8, 3, 7, 4, 9, 2, 6, 5]
```

Start

Think of each element as sorted:

```
[8] [3] [7] [4] [9] [2] [6] [5]
```

---

p = 2

Merge groups of 2:

```
[8] and [3] -> [3, 8]
[7] and [4] -> [4, 7]
[9] and [2] -> [2, 9]
[6] and [5] -> [5, 6]
```

Array becomes:

```
[3, 8, 4, 7, 2, 9, 5, 6]
```

---

p = 4

Merge groups of 4:

```
[3, 8] and [4, 7] -> [3, 4, 7, 8]
[2, 9] and [5, 6] -> [2, 5, 6, 9]
```

Array becomes:

```
[3, 4, 7, 8, 2, 5, 6, 9]
```

---

p = 8

Merge groups of 8:

```
[3, 4, 7, 8] and [2, 5, 6, 9]
```

Array becomes:

```
[2, 3, 4, 5, 6, 7, 8, 9]
```

Now the array is fully sorted.

---

38. Time Complexity of Merging

Merging two sorted parts takes linear time.

If there are `n` total elements in the two parts, merging takes:

```
O(n)
```

Why?

Because each element is copied once into the temporary array and once back into the original array.

The algorithm does not use nested loops to compare every element with every other element.

It simply moves forward through the two sorted parts.

---

39. Time Complexity of Iterative Merge Sort

Iterative Merge Sort has multiple passes.

In each pass, all `n` elements are processed.

Example for 8 elements:

```
Pass 1: process all 8 elements
Pass 2: process all 8 elements
Pass 3: process all 8 elements
```

The number of passes is about:

```
log₂(n)
```

So total time is:

```
O(n log n)
```

Beginner explanation:

- Each pass costs `O(n)`.
- There are about `log n` passes.
- So total cost is `O(n log n)`.

---

40. Space Complexity of Iterative Merge Sort

Iterative Merge Sort needs an auxiliary array for merging.

In the code, this is:

```
int B[100];
```

This extra array stores merged values temporarily.

So the extra space is:

```
O(n)
```

This is different from Bubble Sort, Insertion Sort, and Selection Sort, which use only `O(1)` extra space.

Important idea:

> Merge Sort is faster than simple $O(n^2)$ sorts for large data, but it needs extra memory.

---

41. Is Iterative Merge Sort Stable?

Merge Sort can be stable.

A sorting algorithm is stable if equal values keep their original relative order.

However, stability depends on the comparison used during merging.

In the code shown:

```
if (A[i] < A[j]) {
    B[k++] = A[i++];
} else {
    B[k++] = A[j++];
}
```

If `A[i]` and `A[j]` are equal, the `else` part chooses the right-side value first.

That can break stability.

To make merging stable, use:

```
if (A[i] <= A[j]) {
    B[k++] = A[i++];
} else {
    B[k++] = A[j++];
}
```

This chooses the left-side equal value first.

So:

- Merge Sort is naturally capable of being stable.
- To preserve stability, choose from the left side first when values are equal.

---

42. Is Iterative Merge Sort Adaptive?

In this normal iterative version, Merge Sort is not strongly adaptive.

Even if the array is already sorted, it still performs merging passes.

Example:

```
[2, 3, 4, 5, 6, 7, 8, 9]
```

The array is already sorted.

But Iterative Merge Sort still merges size 1 groups, then size 2 groups, then size 4 groups.

So its time is still usually:

```
O(n log n)
```

It does not stop early like adaptive Bubble Sort.

It does not naturally become `O(n)` like Insertion Sort on sorted data.

---

43. Is Iterative Merge Sort In-Place?

No, not in this implementation.

An in-place sorting algorithm does not need a large extra array.

But Iterative Merge Sort uses an auxiliary array:

```
int B[100];
```

So it is not in-place in the usual sense.

It needs extra memory:

```
O(n)
```

---

44. Properties of Iterative Merge Sort

Property	Iterative Merge Sort result
Sorting type	Comparison-based sorting
Main idea	Merge sorted pieces into bigger sorted pieces
Method	Iterative, loop-based

Property	Iterative Merge Sort result
Uses recursion?	No
Best-case time	`O(n log n)` in the normal version
Average-case time	`O(n log n)`
Worst-case time	`O(n log n)`
Extra space	`O(n)`
In-place?	No, not in this implementation
Stable?	Yes if merging chooses left equal value first using `<=`
Adaptive?	Not strongly adaptive in the normal version
Good for large data?	Yes, usually much better than `O(n²)` sorts

---

45. Comparison With Previous Sorting Algorithms

Algorithm	Main idea	Time complexity	Extra space	Stable?	Adaptive?
Bubble Sort	Push large values to the end	Worst `O(n²)`	`O(1)`	Yes	Yes, with flag
Insertion Sort	Insert into sorted left part	Worst `O(n²)`	`O(1)`	Yes	Yes
Selection Sort	Select minimum and place at front	`O(n²)`	`O(1)`	No	No
Quick Sort	Partition around pivot	Average `O(n log n)`, worst `O(n²)`	Stack space	Usually no	No in first-pivot version
Iterative Merge Sort	Merge sorted pieces	`O(n log n)`	`O(n)`	Yes if implemented carefully	Not strongly adaptive

---

46. Merge Sort vs Quick Sort Idea

Quick Sort and Merge Sort are both powerful sorting algorithms, but they work differently.

Quick Sort idea

Quick Sort says:

> Put one pivot in the correct position, then sort the left and right sides.

Memory trick:

```
partition -> sort left -> sort right
```

Merge Sort idea

Merge Sort says:

> First create small sorted pieces, then merge them into bigger sorted pieces.

Memory trick:

```
small sorted pieces -> bigger sorted pieces -> fully sorted array
```

---

47. Important Difference Between Partitioning and Merging

Concept	Meaning
Partitioning	Places one pivot in its final position
Merging	Combines two sorted parts into one sorted part

Partitioning does not require both sides to be sorted before partitioning.

Merging does require both parts to be sorted before merging.

This is a very important difference.

---

48. Common Beginner Mistakes in Phase 7

Mistake 1: Trying to merge unsorted parts

Wrong idea:

> I can take any two random sections and merge them into sorted order.

Correct idea:

> Merge works only when both sections are already sorted.

Mistake 2: Forgetting to copy leftovers

Wrong code idea:

```
while (i <= mid && j <= high) {
    // compare and copy
}
```

Then stopping.

Problem:

One side may still have values left.

Correct code:

```
while (i <= mid) {
    B[k++] = A[i++];
}

while (j <= high) {
    B[k++] = A[j++];
}
```

Mistake 3: Forgetting to copy B back into A

After merging, values are in `B`.

If we forget this:

```
for (i = low; i <= high; i++) {
    A[i] = B[i];
}
```

then the original array `A` will not be updated.

Mistake 4: Using wrong mid value

This line is important:

```
mid = (low + high) / 2;
```

If `mid` is wrong, the program may merge the wrong sections.

Mistake 5: Confusing p with index i

`p` is not an array index.

`p` means the current merge group size.

Examples:

```
p = 2
p = 4
p = 8
```

`i` is used to move through the array.

Mistake 6: Ignoring array sizes that are not powers of 2

If the array size is not exactly:

```
2, 4, 8, 16, ...
```

then some elements may be left over during passes.

The final `if` handles that case:

```
if (p / 2 < n) {
    Merge(A, 0, p / 2 - 1, n - 1);
}
```

Mistake 7: Thinking Iterative Merge Sort uses recursion

Recursive Merge Sort uses recursion.

Iterative Merge Sort uses loops.

Phase 7 is about Iterative Merge Sort.

Phase 8 will cover Recursive Merge Sort.

Mistake 8: Thinking Merge Sort is in-place

This version uses extra array `B`.

So it is not in-place.

Extra space is:

```
O(n)
```

---

49. Complete Phase 7 Program in C

```c
#include <stdio.h>

void Merge(int A[], int low, int mid, int high) {
    int i = low;
    int j = mid + 1;
    int k = low;
    int B[100];

    while (i <= mid && j <= high) {
        if (A[i] < A[j]) {
            B[k++] = A[i++];
        } else {
            B[k++] = A[j++];
        }
    }

    while (i <= mid) {
        B[k++] = A[i++];
    }
```

```
    while (j <= high) {
        B[k++] = A[j++];
    }

    for (i = low; i <= high; i++) {
        A[i] = B[i];
    }
}

void IterativeMergeSort(int A[], int n) {
    int p, i, low, mid, high;

    for (p = 2; p <= n; p = p * 2) {
        for (i = 0; i + p - 1 < n; i = i + p) {
            low = i;
            high = i + p - 1;
            mid = (low + high) / 2;
            Merge(A, low, mid, high);
        }
    }

    if (p / 2 < n) {
        Merge(A, 0, p / 2 - 1, n - 1);
    }
}

int main() {
    int A[] = {8, 3, 7, 4, 9, 2, 6, 5};
    int n = sizeof(A) / sizeof(A[0]);

    IterativeMergeSort(A, n);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", A[i]);
    }

    printf("\n");
    return 0;
}
```

Expected output:

```
Sorted array: 2 3 4 5 6 7 8 9
```

---

50. Program Explanation Summary

The program has three main parts:

1. `Merge()` function
2. `IterativeMergeSort()` function
3. `main()` function

Merge() function

The `Merge()` function combines two sorted sections of the same array.

It uses:

- `i` for the left section.
- `j` for the right section.
- `k` for the temporary array.
- `B[]` as the auxiliary array.

It compares values, copies the smaller one, copies leftovers, and then copies `B` back to `A`.

IterativeMergeSort() function

The `IterativeMergeSort()` function controls the passes.

It uses `p` to control group size.

`p` starts at `2` and doubles each time:

```
2, 4, 8, 16, ...
```

Each pass merges smaller sorted groups into bigger sorted groups.

---

main() function

The `main()` function:

1. Creates the array.
2. Calculates `n`.
3. Calls `IterativeMergeSort()`.
4. Prints the sorted array.

51. Phase 7 Key Takeaways

- Merging means combining two already sorted lists into one sorted list.
- Merging does not work correctly on random unsorted chunks.
- Two separate sorted arrays can be merged into a third array.
- Two sorted sections inside one array can be merged using an auxiliary array.
- `i`, `j`, and `k` are the main movement variables in merging.
- `low`, `mid`, and `high` describe the two sorted sections inside one array.
- Leftover elements must be copied after one side finishes.
- The merged result must be copied back from `B` to `A`.
- 2-way merging means merging two sorted lists.
- m-way merging means merging many sorted lists.
- Iterative Merge Sort starts with one-element sorted lists.
- It merges groups of size `1` into `2`, `2` into `4`, `4` into `8`, and so on.
- `p` represents the current merge group size.
- Iterative Merge Sort uses loops, not recursion.
- Time complexity is `O(n log n)`.
- Space complexity is `O(n)`.
- It can be stable if equal values are taken from the left side first.
- It is not in-place in this version because it uses an auxiliary array.

---

52. Final Beginner Memory Trick

Remember Phase 7 like this:

```
[one-element sorted lists]
        ↓
merge into sorted pairs
        ↓
merge into sorted groups of 4
        ↓
merge into sorted groups of 8
        ↓
whole array sorted
```

Short version:

> 1 becomes 2, 2 becomes 4, 4 becomes 8.

Even shorter:

> Sorted pieces combine.

53. How Phase 7 Prepares You for Phase 8

Phase 7 teaches Iterative Merge Sort.

It uses loops to create sorted groups and merge them.

Phase 8 will teach Recursive Merge Sort.

Recursive Merge Sort uses the same merging idea, but it creates the smaller sorted parts using recursion.

So the most important thing to carry into Phase 8 is:

> Merge Sort is built on the Merge function.

If you understand merging, Recursive Merge Sort becomes much easier.